

Title: (Extending the hands-on project) Analyzing Netflix Encryptions

Problem: Once a Netflix session is wrapped in TLS, a network operator sees only packet sizes and timestamps. It cannot tell when the player asked for a new piece of video, what quality tier it landed on, or whether it stalled. Yet those are exactly the things an ISP needs to diagnose a bad stream.

Adaptive streaming works in fixed-length chunks (Netflix's are roughly 4 seconds). If you can recover the moment each chunk was requested, the bytes that arrive between two requests are one chunk, and chunk size divided by chunk duration gives the encoded bitrate, which maps almost directly to resolution (about 0.2 MB per chunk at 480p, about 1 MB at 1080p, about 8 MB at 4K). Raw throughput cannot do this: in the hands-on trace it swung from 1.3 to 6.4 Mbps while bytes per chunk sat near 1 MB and request spacing bounced between 2, 4, and 6 seconds. Throughput mixes video quality with network speed, and chunk boundaries separate the two.

Why does it matter? A model that recovers playback state from encrypted traffic lets operators fix streaming problems without inspecting payloads. That removes the main practical argument for weakening encryption, so the privacy case and the operational case point the same direction.

Concrete question. Can a classifier trained on lightweight, per-window traffic features locate chunk requests in an encrypted Netflix session more accurately than a simple idle-gap heuristic, and does the resulting bitrate estimate hold up?

Data:

1. Primary: netflix.pcap / netflix.csv.gz from the hands-on project. One live Netflix session captured 11 February 2018 in Paris. 141,471 packets over 495.8 seconds, single client 192.168.43.72. The first minute contains non-Netflix traffic (Google, YouTube, DoubleClick, Akamai) that must be filtered out using TLS SNI and Netflix Open Connect IP ranges.
2. Validation (stretch): the labeled video-session dataset released with Bronzino et al., "Inferring Streaming Video Quality from Encrypted Traffic" (SIGMETRICS 2020), which includes ground-truth resolution labels across Netflix, YouTube, Amazon, and Twitch. This checks that the single-session findings are not a fluke of one capture.

Label derivation. The pcap has no labels. Chunk requests will be labeled from the uplink direction: a client-to-server TLS record carrying application data (a few hundred bytes) that follows an idle period marks a request. The model will then be trained on downlink-only features, so it has to learn the request rhythm from the server side rather than reading the answer off the uplink. This mirrors an operator who only sees an asymmetric or aggregated view of the flow.

Features. Per 250 ms window, from downlink packets only: byte count, packet count, mean and max packet size, inter-arrival mean and variance, time since the last empty window, rolling byte totals over 1 s and 4 s, and number of active server connections.

Machine Learning:

- Baseline (no ML): flag a request whenever downlink traffic resumes after an idle gap longer than a threshold. The ML models must beat this or the project has no reason to use them.
- L1 logistic regression: linear and interpretable with built-in feature selection, needs scaling. Expected to be the weakest.
- Random forest: robust to untuned settings and correlated features, needs class weighting.
- Gradient-boosted trees (expected best): captures interactions such as "large byte burst after long idle" and reports feature importance, but overfits unless depth and learning rate are kept small.

Evaluation

1. Request detection. Precision, recall, and F1 on the request-window class, with a tolerance of plus or minus one window so an off-by-one prediction counts as a hit. Confusion matrix reported for each model and the baseline.
2. Bitrate recovery. Using predicted requests, compute bytes per chunk divided by chunk duration, then compare the recovered bitrate series against the series built from the uplink-derived labels. Mean absolute error in kbps and agreement on resolution tier.
3. Split strategy. This is a time series, so random K-fold would leak. Use blocked, time-ordered folds (5 contiguous blocks, train on four, test on one) and report the spread across folds, not just the mean.
4. Stretch. Retrain the winning model on the Bronzino labeled data and report resolution-tier accuracy against real ground truth.

Learning Objectives:

1. Deriving usable labels from a dataset that has none, and being honest about the circularity risk that comes with it.
2. Measuring exactly what TLS leaks about application behavior, and what it does not.
3. Working with a small, autocorrelated dataset (a few hundred windows, about 120 positives) without being fooled by a bad split.
4. Building a non-ML baseline and only claiming ML is useful when it actually beats that baseline.

Group members: Ernie, Angelina, and Yosan